

Codex Planning Handoff — Free/Open-Source Offline Survival App

0. Purpose of This Handoff

This document captures the important implementation-relevant context from a planning conversation about building a free/open-source Android-first survival app.

The reader should assume they have **not** seen the original chat. This handoff preserves the project goal, architecture direction, product decisions, delayed ideas, constraints, edge cases, and immediate next steps.

This is intended to be uploaded into Codex alongside two related branch conversations so Codex can continue planning and implementation without repeating old work or making wrong assumptions.

1. Project Goal

Core Goal

Build the best possible **free/open-source survival app** for outdoors people, preppers, hikers, backpackers, overlanders, amateur radio users, disaster-preparedness communities, and eventually SAR/CERT-style users.

The product should be:

- Offline-first
- Android-first for MVP
- Free and open source
- Useful without AI
- Enhanced by optional AI/RAG later
- Lightweight at install
- Modular through downloadable packs
- Built publicly to attract community, contributors, testers, and early adopters

The long-term ambition is to create a product that users would not only install, but also **share**, because it is genuinely useful in offline, wilderness, emergency, and grid-down contexts.

Product Positioning

The app should not just be another survival manual.

The wedge is:

A polished, offline-first survival app that combines local maps, location-aware search, guided survival knowledge, provenance/citations, and user-controlled downloadable packs.

The app should feel more like a **field instrument** than a chatbot or generic prepper PDF reader.

2. Current State

No Code Yet From This Chat

No repository, files, scripts, commands, or app implementation were created in this conversation.

Current status is **product/architecture planning**.

Main Decisions Made

The plan converged toward:

1. **Android-first app**
 2. **Free/open-source**
 3. **Offline-first**
 4. **Small base app**
 5. **Marketplace-style downloadable packs**
 6. **OpenStreetMap-based offline map data**
 7. **Local geospatial query layer**
 8. **Guided RAG/chat later**
 9. **Avoid broad “ask me anything” behavior**
 10. **Avoid medical-heavy MVP**
 11. **Build in public**
 12. **Weekly public updates**
 13. **GitHub as source-of-truth**
 14. **Twitter/X and possibly Reddit for public progress updates**
 15. **Field notes and peer-to-peer sharing as strong future differentiators**
 16. **Do not share peer-to-peer content packs initially because pack ingestion/generation may remain proprietary**
-

3. Market Context and Competitive Gap

A previous research pass found that the survival/offline-preparedness market has many partial solutions, but few integrated ones.

Existing Strong Products by Category

The current market has strong products in isolated layers:

- **Trail Sense:** strong offline field tools
- **Organic Maps:** strong offline maps/navigation
- **OsmAnd:** strong offline map/routing/navigation stack
- **Kiwix:** strong offline libraries
- **Meshtastic:** off-grid radio/mesh communication
- **Briar:** phone-to-phone/offline-ish secure messaging
- **SurvivalRAG / similar prototypes:** source-cited local Q&A or RAG-style retrieval

Market Gap

The gap is not “more content.”

The bigger gap is:

Integrated offline survival UX + maps + local resource search + guided survival guidance + provenance + lightweight mobile-first execution.

Most apps do one layer well. Few combine the layers cleanly.

Differentiation Opportunity

A strong wedge would be:

- Offline map/location layer
- Local geospatial resource queries
- Guided survival workflows
- Cited/provenance-first answer cards
- Lightweight downloadable packs
- Optional local AI/RAG
- Build-in-public open-source community

The opportunity is especially strong if the app avoids looking like a hacker project and instead feels like a clean, practical outdoor/off-grid tool.

4. Final Product Direction

Product Concept

A modular offline survival app with four primary tabs:

1. **Map**
2. **Tools**
3. **Guide**
4. **Packs**

The app should be useful before AI is added.

The AI/RAG layer should enhance the app, not be required for the app to function.

Core User Story

A user is offline in the woods and says something like:

"I'm thirsty. I need water."

The system should eventually:

1. Read the user's GPS location.
2. Query local/offline map data for nearby water sources.
3. Return likely nearby water-related features.
4. Retrieve survival guide cards about finding, purifying, and storing water.
5. Show a guided response with citations/provenance.
6. Avoid unsupported claims.
7. Clearly separate map data from survival guidance.

Example response behavior:

- "Mapped stream 0.4 miles northwest"
 - "Mapped lake 1.2 miles east"
 - "Purification checklist: boil, filter, disinfect, store"
 - "Source: Water Purification Pack v1.0"
 - "Map data source: OpenStreetMap"
 - "Warning: mapped water may be seasonal or dry; verify in field"
-

5. Major Feature Pillars

Pillar 1 — Offline Maps / Location Layer

The app should use offline map data, likely based on OpenStreetMap.

Important clarification:

- GPS itself works offline.
- The phone can receive satellite signals without internet.
- But maps, trails, water sources, POIs, topography, roads, and infrastructure data must be downloaded ahead of time.

The app should allow users to download only the map/data packs they want.

Pillar 2 — Local Geospatial Query

The app should support local queries over downloaded map/resource data.

Examples:

- Nearest water source
- Nearest road
- Nearest trailhead
- Nearest shelter
- Nearest hospital
- Nearest ranger station
- Nearest gas station
- Nearest bridge
- Nearest high ground
- Nearest mapped infrastructure/resource

This does **not** require AI for MVP.

Core local search should use conventional geospatial indexing and spatial queries.

Pillar 3 — Guided Survival Knowledge / RAG

The survival knowledge interface should not be a generic “ask me anything” chatbot.

It should be guided.

Examples:

- “I need water”

- “I’m lost”
- “I need shelter”
- “I need fire”
- “I need to signal”
- “I need to navigate back”
- “I need to preserve battery”
- “I need to store water”
- “I need to prepare for cold weather”

The app should retrieve relevant guide cards/checklists and optionally use a local model to summarize or compose an answer.

Pillar 4 — Provenance and Trust

Every guidance answer should make source/provenance visible.

The app should clearly separate:

- Map data
- Source excerpts
- App-generated synthesis
- User/community field notes
- Model-generated text
- Confidence level or uncertainty

Do not hide the source of advice.

Pillar 5 — Marketplace-Style Packs

The app should have a lightweight base install.

Users should download packs relevant to them.

Pack examples:

- Pennsylvania map pack
- Allegheny National Forest pack
- Wilderness water pack
- Fire-starting pack
- Shelter pack
- Navigation basics pack
- Urban preparedness pack
- Amateur radio/comms pack
- Overlanding pack
- Winter survival pack
- Desert survival pack
- Storm prep pack

The app should not require a giant full-country download.

Users can download broader packs if they want.

Pillar 6 — Field Notes and Peer-to-Peer Sharing

Field notes are a strong differentiator.

The app should eventually support:

- Local field notes
- Notes attached to GPS locations
- Notes attached to waypoints
- Notes attached to checklists
- Peer-to-peer sharing of notes
- Possibly signed/tamper-evident notes later
- QR/Bluetooth/Wi-Fi Direct sharing later

Decision:

- Peer-to-peer field notes are good.
 - Peer-to-peer survival/content pack sharing should be delayed or avoided initially because content ingestion/generation may stay proprietary.
-

6. Offline Maps: Key Findings and Constraints

GPS Works Offline

The device can access GPS satellite information without internet.

However:

- GPS gives position.
- It does not give map context.
- It does not provide trails, water, roads, shelters, POIs, or topography.
- Those must be downloaded or bundled separately.

OpenStreetMap

The first public map data source discussed was:

OpenStreetMap

OpenStreetMap is an open geographic data project used by many offline mapping apps.

It can contain:

- Roads
- Trails
- Paths
- Rivers
- Streams
- Lakes
- Buildings
- Points of interest
- Campgrounds
- Shelters
- Hospitals
- Shops
- Fuel/gas
- Terrain-related map features depending on region/tagging

Important warning:

- OpenStreetMap quality varies by area.
- Data may be incomplete, outdated, or unverified.
- The app must not imply certainty.
- UX should say “mapped water source,” not guaranteed water source.

How OSM Data Is Stored

OpenStreetMap data can be stored/exported as:

- OSM XML
- PBF / Protocolbuffer Binary Format
- Preprocessed vector tiles
- MBTiles
- App-specific map files
- Local SQLite databases
- Spatial indexes

For app use, raw OSM is usually preprocessed.

Querying OSM/Map Data

Nearest-feature queries should not rely only on FTS.

Use:

- Spatial indexes
- Coordinates
- Bounding boxes

- R-tree style indexes
- Distance calculations
- Optional FTS for names/categories/text labels

FTS is useful for:

- Searching names
- Searching POI labels
- Searching text fields
- Searching guide cards

Spatial indexing is required for:

- "nearest"
- "within radius"
- "near me"
- "distance to"
- "bearing to"
- "features around current location"

Offline Maps Are Challenging Because

Offline maps involve many systems:

1. Map data
2. Rendering
3. Offline search
4. Offline routing
5. Downloads
6. Updates
7. Licensing
8. Storage management
9. Battery management
10. GPS/compass/sensor behavior
11. Topographic data
12. Field usability

Avoid Building a Full Map App First

The app should not try to immediately compete with Organic Maps or OsmAnd as a full navigation engine.

The MVP should not build full routing.

Start with:

- GPS coordinates
- Waypoints

- Distance and bearing
 - Breadcrumb/backtrack
 - Offline map/resource packs
 - Simple local resource queries
 - External map app handoff if needed
-

7. MVP Scope

MVP Should Not Start as a Full AI App

The core MVP can work without AI.

AI is not needed for:

- Offline maps
- Waypoints
- GPS
- Distance/bearing
- Spatial search
- Saved notes
- Checklists
- Pack installation
- Basic guide cards

AI can come later for:

- Natural language interpretation
- Guided responses
- Summarization
- RAG answer composition
- Context-aware recommendations

MVP Product Definition

The MVP should be:

An Android-first offline survival/location app with local map/resource packs, basic geospatial search, waypoints, checklists, guide cards, and pack management.

MVP Tabs

1. Map Tab

MVP features:

- Request location permission
- Show current latitude/longitude
- Show GPS accuracy
- Show altitude if available
- Show heading/bearing if available
- Save current location as waypoint
- View saved waypoints
- Calculate distance to selected waypoint
- Calculate bearing to selected waypoint
- Breadcrumb recording
- Backtrack mode
- Optional map viewer if simple enough
- Open location in external map app through geo URI
- Offline readiness status for downloaded map/location data

Do **not** require full offline routing in MVP.

2. Tools Tab

MVP features:

- Compass/bearing tool if available
- Timer
- Flashlight shortcut/action placeholder
- Coordinate display
- Checklist access
- Battery/emergency mode placeholder
- Possibly signal/whistle/flash pattern later

3. Guide Tab

MVP features:

- Searchable survival cards
- Guided categories rather than open chat
- Water
- Fire
- Shelter
- Navigation
- Signaling
- Cold/weather

- Food basics
- Safety disclaimers
- Provenance/source shown on each card

4. Packs Tab

MVP features:

- Installed packs list
 - Pack metadata
 - Import local pack file
 - Enable/disable packs
 - Delete packs
 - Show storage used
 - Show offline readiness
 - Show pack version/license/source
 - Later: remote registry/marketplace
-

8. Content Packs

Pack Strategy

Use a marketplace-style system.

Base app remains small.

Users download what they want.

This fits prepper/outdoor use because different users want different content:

- One user may want Pennsylvania maps.
- Another wants desert survival.
- Another wants overlanding.
- Another wants amateur radio.
- Another wants urban preparedness.

Pack Types

Possible pack types:

1. **Map/region packs**
2. **Resource/POI packs**
3. **Survival guide packs**
4. **Checklist packs**

5. Field workflow packs
6. Community field note packs later
7. Optional RAG/vector index packs later

Pack Metadata

Each pack should include a manifest with metadata.

Suggested fields:

```
{
  "pack_id": "water_basics_v1",
  "title": "Water Basics",
  "version": "1.0.0",
  "category": "water",
  "source_name": "Example Source",
  "source_url": "https://example.com",
  "license": "CC-BY-4.0",
  "last_reviewed": "2026-06-30",
  "content_type": "guide",
  "offline_ready": true,
  "files": []
}
```

For map/resource packs:

```
{
  "pack_id": "pa_osm_resources_v1",
  "title": "Pennsylvania Offline Resource Pack",
  "version": "1.0.0",
  "category": "map_resources",
  "data_sources": [
    {
      "name": "OpenStreetMap",
      "license": "ODbL",
      "attribution_required": true,
      "last_updated": "2026-06-01"
    }
  ],
  "includes": [
    "roads",
    "trails",
    "water",
    "shelters",
    "hospitals",
  ]
}
```

```
    "fuel",  
    "campgrounds"  
  ],  
  "offline_ready": true  
}
```

Important Decision

Do **not** share peer-to-peer packs initially.

Reason:

- Pack creation may require ingestion pipelines.
- Pack generation may become proprietary or controlled.
- Peer-to-peer pack sharing creates licensing, trust, security, and moderation issues.

Instead:

- Allow peer-to-peer field note sharing later.
- Keep official/curated packs controlled and versioned.

9. Guided Chat / RAG Direction

Avoid Free-Form “Ask Me Anything”

The app should not start as a broad chatbot.

Survival contexts are high risk.

Guided chat reduces:

- Hallucinations
- Unsafe advice
- Liability
- User confusion
- Poor retrieval
- Overly broad answers

Guided Interaction Model

The app should route user intent into known workflows.

Example categories:

- Water
- Shelter
- Fire
- Navigation
- Signaling
- Weather/cold
- Food
- Gear/kit
- Lost person/self-rescue
- Battery conservation
- Urban prep
- Communications

Example Query Flow

User says:

"I'm thirsty and I'm in the middle of the woods."

System flow:

1. Query router detects:
2. Need: water
3. Context: wilderness
4. Urgency: possible dehydration
5. Location needed: yes
6. Pull current GPS location.
7. Run local geospatial query:
8. nearest streams
9. nearest lakes
10. nearest springs
11. nearest water-related POIs
12. Run local guide/RAG query:
13. finding water
14. purifying water
15. storing water

16. Compose guided response:
17. nearest mapped water options
18. distance/bearing
19. caution about map data reliability
20. purification checklist
21. storage checklist
22. source/provenance

Parallel Retrieval

Geospatial query and RAG retrieval should run in parallel.

Expected latency:

- Local geospatial query: usually milliseconds to low hundreds of ms
- Local keyword/FTS retrieval: fast
- Local vector retrieval: depends on index size but should be fast if compact
- Local model generation: likely main latency source
- Total local AI answer latency: possibly a few seconds depending on device/model

Important:

- Do not block geospatial results on model generation.
- Show immediate structured results first when possible.
- The app can display “nearest mapped water sources” before final guided explanation finishes.

10. AI / RAG Architecture

AI Not Required for MVP

The app should first work as a deterministic offline survival/location tool.

RAG v1

Later RAG should include:

- Curated packs only
- Chunked documents/cards
- Local index
- Citations
- Source excerpts
- Confidence/provenance UI

- Guided answer composition

Context Injection

The RAG/context system should inject geolocation context into the prompt/context window.

Example context:

```
Current location:
- Latitude: 40.12345
- Longitude: -79.12345
- Accuracy: ±12m
- Elevation: 421m

Nearby mapped resources:
1. Stream, 0.4 mi NW, source: OpenStreetMap
2. Lake, 1.2 mi E, source: OpenStreetMap
3. Campground water point, 2.3 mi S, source: OpenStreetMap

User request:
"I'm thirsty. I need water."

Retrieved guide cards:
- Water Purification Basics
- Field Water Collection
- Water Storage Safety
```

The model should then produce a guided answer grounded in both map/resource data and survival content.

Query Router / Small Agent

A simple local “agent” or query router may be needed.

Responsibilities:

- Classify user intent
- Decide whether geolocation is needed
- Decide which map/resource query to run
- Decide which RAG guide category to search
- Avoid answering outside supported workflows
- Return fallback if no data is downloaded

This does not need to be a complex autonomous agent.

It can start as rules/classifiers.

11. Medical Content Decision

Concern

Medical content was flagged as risky.

Reason:

- Doctors/medical professionals can poke holes in bad or oversimplified guidance.
- A hallucination or incorrect medical instruction could severely damage credibility.
- Medical advice carries higher liability and trust risk than general survival skills.

Decision

Avoid medical-heavy MVP.

Do not lead with medical packs.

Possible later approach:

- Add limited “first aid quick cards” only after careful review.
- Use conservative, sourced, non-diagnostic content.
- Clearly instruct users to seek professional medical help when available.
- Keep medical content guided and heavily cited.
- Possibly exclude advanced medical procedures entirely.

Better Initial Packs

Start with lower-risk, high-value survival domains:

1. Water finding
2. Water purification
3. Water storage
4. Shelter
5. Fire
6. Navigation basics
7. Signaling
8. Weather/cold exposure preparation
9. Gear/kit checklists
10. Food safety basics, but avoid dangerous foraging confidence claims early

12. Emergency Mode

A possible missing feature identified was **Emergency Mode**.

This could be a non-AI differentiator that does not require peer-to-peer pack sharing.

Emergency Mode Ideas

- Offline SOS checklist
- "I am lost" flow
- "I need water" flow
- "I need shelter" flow
- "I need to signal" flow
- "Preserve battery" mode
- Flashing screen signal
- Audio signal/whistle pattern
- Coordinates card
- Share location by QR/text
- Last known location
- Backtrack
- Emergency notes
- Minimal UI with large buttons
- Disable AI unless explicitly requested to save battery

Recommended Emergency Mode MVP

Start with:

- "I'm lost"
- "Need water"
- "Need shelter"
- "Signal for help"
- "Save battery"
- "Show my coordinates"
- "Backtrack"

13. Field Notes and Peer-to-Peer Sharing

Field Notes Are Valuable

Field notes can create a useful network effect without requiring official content pack sharing.

Examples:

- "Bridge washed out"
- "Stream dry in summer"
- "Good campsite here"
- "Water filter clogged here"
- "Ranger station closed"
- "Trail blocked"
- "Cell signal here"
- "Vehicle parked here"
- "Cache location"
- "Unsafe area"
- "Good radio contact point"

Local Field Notes MVP

Initial field notes can be simple:

- Text note
- Category
- GPS coordinate
- Timestamp
- Optional photo later
- Attached waypoint
- Attached checklist
- Private by default

Peer-to-Peer Field Notes Later

Potential sharing methods:

- QR code
- Bluetooth
- Wi-Fi Direct
- Local LAN
- Meshtastic bridge later

Signed Notes Later

A later trust feature:

- Signed/tamper-evident field notes
- Expiry date
- Author/device identity
- Warning if untrusted
- "Community note — not verified" label

14. Build-in-Public Strategy

Decision

Do not build privately and then release.

Announce the project early and build in public.

Platforms

Use multiple platforms, but with clear roles.

GitHub

Primary source of truth.

Use for:

- Code
- Issues
- Roadmap
- Discussions
- Releases
- Contributor onboarding
- Documentation
- Pack schema
- License
- Build instructions

Twitter/X

Use for weekly progress posts.

Post:

- Screenshots
- Short demos
- Architecture decisions
- Feature clips
- Build logs
- “What should we build next?” questions
- Milestone updates

Reddit

Use carefully for feedback and community validation.

Potential subreddits:

- r/preppers
- r/Survival
- r/CampingandHiking
- r/Ultralight
- r/amateurradio
- r/overlanding
- r/opensource
- r/androidapps
- r/fossdroid

Avoid spam. Ask for feedback, not promotion.

Cadence

Weekly updates are recommended.

A weekly cadence is frequent enough to show progress but gives enough time to build something meaningful.

Build-in-Public Content Loop

Suggested loop:

1. Build feature
2. Post screenshot/demo
3. Explain why it matters offline
4. Ask for field feedback
5. Capture feedback into GitHub issues
6. Ship next improvement
7. Repeat weekly

Community Involvement

Ways to get people involved:

- Public roadmap
- GitHub Discussions
- Good first issues
- Beta tester signup
- Pack idea requests

- Field testing checklist
 - Device compatibility reports
 - “Submit your offline survival workflow”
 - “What would you need if you lost service in the woods?”
-

15. Footprint / Storage Strategy

Base App

Base app should be small.

Target:

- Lightweight initial install
- No giant bundled map files
- No large AI model in base install
- No huge PDFs in base install

Rough target:

- Base app ideally under ~100 MB if possible
- Larger optional downloads handled through packs

User-Controlled Downloads

Users should download only what they want.

This fits a marketplace-style approach.

Examples:

- “Download Pennsylvania map resources”
- “Download Water Pack”
- “Download Fire Pack”
- “Download Winter Pack”
- “Download Amateur Radio Pack”
- “Download Allegheny National Forest pack”

Offline Readiness

The app should show whether the user is actually prepared offline.

Example:

Offline readiness: 72%

Downloaded:

- ✓ Base survival guide
- ✓ Water pack
- ✓ Fire pack
- ✓ Local waypoints
- ✓ Pittsburgh/Allegheny map resources

Missing:

- △ Topographic layer
- △ Nearby water source layer
- △ Offline routing graph
- △ Shelter pack

This is a major UX differentiator.

16. Architecture and Stack Choices Discussed

Confirmed Direction

- Android-first
- Offline-first
- Free/open-source
- Local storage
- Downloadable packs
- OpenStreetMap-based data
- Local spatial queries
- Guided RAG later
- Public GitHub repo
- Build in public

Candidate/Discussed Technical Components

Mobile Platform

- Android-first
- Kotlin likely, but not formally decided in this chat
- Native Android preferred for field-device reliability

Local Storage

Discussed/suggested:

- SQLite
- SQLite FTS for text/card search
- R-tree/spatial indexing for geospatial queries
- SQLCipher or encrypted local storage later for sensitive notes

Map/Data Formats

Discussed:

- OpenStreetMap
- OSM XML
- PBF / Protocolbuffer Binary Format
- MBTiles
- Vector map formats
- Local map/resource packs

Query Methods

- FTS for text search
- Spatial index for nearest/within-radius queries
- Distance and bearing calculations
- Semantic/vector retrieval later for RAG
- Query router/agent for human-centric user intent

AI/RAG

Discussed conceptually:

- Local RAG
- Local index over curated content packs
- Guided chat
- Geolocation injected into context window
- Parallel geospatial + RAG search
- Optional local model later
- No specific production model chosen in this chat

Earlier broader roadmap context mentioned possible model classes:

- Small phone-tier model
- Tablet/laptop tier model
- LAN/basecamp host model
- Do not hard-wire one model family

No final model decision was made in this chat.

APIs/External Apps

- `geo:` URI / external map app handoff was suggested for MVP
- Organic Maps/OsmAnd can be used externally rather than duplicating full routing immediately

Tools Mentioned

- GitHub
- Twitter/X
- Reddit
- Codex
- OpenStreetMap
- Organic Maps
- OsmAnd

No concrete scripts, repo names, package names, build commands, or file paths were established.

17. Explicit Ideas Rejected or Delayed

Rejected/Delayed: Full Offline Routing in MVP

Do not build turn-by-turn offline routing first.

Reason:

- Routing graph is complex.
- Trail/road permissions are complex.
- Walking/driving profiles are complex.
- Elevation and surface penalties add complexity.
- Existing map apps already do this well.

Start with:

- Coordinates
- Waypoints
- Bearing
- Distance
- Breadcrumb/backtrack
- External map handoff

Rejected/Delayed: Full Map App Competitor

Do not try to beat Organic Maps/OsmAnd immediately.

The app should be a survival app that uses map intelligence, not a pure map/navigation app.

Rejected/Delayed: Medical-Heavy MVP

Avoid medical content early.

Reason:

- High trust/liability risk
- Doctors can challenge it
- Hallucination could sabotage the project

Rejected/Delayed: Peer-to-Peer Pack Sharing

Do not share official content packs peer-to-peer initially.

Reason:

- Pack ingestion/generation pipeline may remain proprietary.
- Licensing and trust are difficult.
- Bad packs could damage reputation.

Field-note sharing is safer and more useful early.

Rejected/Delayed: Free-Form Chatbot

Do not start with an open-ended chatbot.

Use guided workflows.

Rejected/Delayed: Bundling All Map Data by Default

Users may theoretically download all of America, but base app should not include giant datasets.

Use optional regional downloads.

18. Important Warnings and Edge Cases

Map Data Reliability

OpenStreetMap data may be incomplete or wrong.

Never state:

“There is water here.”

Prefer:

“Mapped water feature nearby. Verify in field.”

Seasonal Water Sources

Streams/springs may be dry.

Need disclaimers and confidence labels.

GPS Limitations

GPS may fail or degrade:

- Indoors
- Canyons
- Dense forest
- Bad weather/terrain
- Low battery
- Device sensor issues

Show GPS accuracy clearly.

Battery

Maps + GPS + AI can drain battery.

Need:

- Battery saver
- Emergency mode
- GPS polling controls
- AI off by default in battery saver
- Dim/minimal map mode
- Coordinate-only mode

Storage

Map and content packs can become huge.

Need:

- Storage estimates before download
- Delete pack
- Update pack
- Pack storage breakdown

- Offline readiness screen

Licensing

Map/data/content licenses matter.

Need:

- OSM attribution
- Pack licenses
- Source metadata
- License sidecars/manifests
- Clear attribution UI

Trust

Survival guidance must show sources.

Avoid unsupported model guesses.

AI Hallucination

Do not let the model invent nearby resources.

Nearby resource results must come from local spatial data.

Do not let the model create medical advice beyond source content.

No Internet Assumption

The app must remain useful when:

- No cellular
- No Wi-Fi
- No cloud
- No API calls
- No account login

19. Recommended Development Roadmap

Phase 0 — Repository and Planning Setup

Goal: prepare the open-source project before heavy coding.

Create:

- GitHub repo
- README
- LICENSE
- CONTRIBUTING
- CODE_OF_CONDUCT
- ROADMAP
- SECURITY
- docs/ folder
- issue templates
- discussion categories
- pack schema draft
- public project board

Recommended initial repo description:

Offline-first open-source survival companion for maps, field tools, guide packs, and location-aware survival workflows.

Phase 1 — Non-AI MVP

Goal: useful without AI.

Features:

- Android app shell
- Four tabs: Map, Tools, Guide, Packs
- Local SQLite storage
- Save/view waypoints
- Current GPS coordinates
- Distance/bearing to waypoint
- Breadcrumb track recording
- Backtrack mode
- Static guide cards
- Checklist support
- Pack manifest parser
- Installed packs list
- Local import of packs
- Basic FTS search over guide cards
- Provenance display on guide cards
- Offline readiness screen

Phase 2 — Offline Map/Resource Pack MVP

Goal: local resource search.

Features:

- OpenStreetMap-derived resource pack prototype
- Local spatial index
- Query nearest water
- Query nearest road/trail/shelter/etc.
- Show distance/bearing
- Show map-data provenance
- Show uncertainty
- Optional map viewer if feasible
- External app handoff

Phase 3 — Guided Survival Workflows

Goal: no AI yet, but smarter UX.

Features:

- “I need water”
- “I’m lost”
- “Need shelter”
- “Need fire”
- “Need to signal”
- “Save battery”
- Guided checklist flows
- Combine nearby resources + guide cards
- Show step-by-step response without LLM

Phase 4 — RAG v1

Goal: local cited retrieval over curated packs.

Features:

- Chunk guide packs
- Local retrieval index
- Keyword + semantic retrieval
- Cited answer cards
- Source excerpts
- Confidence/provenance UI
- Optional local model
- Run geospatial + RAG retrieval in parallel

Phase 5 — Field Notes

Goal: user-generated local intelligence.

Features:

- Field notes attached to waypoint
- Categories
- Timestamp
- Local-only storage
- Export/import notes
- Later QR/Bluetooth/Wi-Fi Direct sharing
- Later signed/tamper-evident notes

Phase 6 — Community / Pack Registry

Goal: marketplace-style pack discovery.

Features:

- Pack registry
- Pack categories
- Download/update/delete
- Pack ratings or verification later
- Community pack suggestions
- Official pack review workflow

20. Immediate Next Steps for Codex

Step 1 — Create the Repo Skeleton

Create the project structure and docs.

Suggested repo name options:

- offline-survival-companion
- open-survival
- field-companion
- survival-layer
- unplugged-survival
- survival-os

No final repo name was chosen.

Step 2 — Create Product Docs

Create:

```
README.md
ROADMAP.md
CONTRIBUTING.md
CODE_OF_CONDUCT.md
LICENSE
docs/architecture.md
docs/pack-schema.md
docs/offline-readiness.md
docs/map-data-strategy.md
docs/rag-strategy.md
docs/build-in-public.md
```

Step 3 — Build Android App Skeleton

Initial app:

- Four tabs: Map, Tools, Guide, Packs
- Local SQLite database
- Dummy guide pack bundled
- Dummy pack manifest
- Local guide card search
- Waypoint save/list
- GPS coordinate screen
- Offline readiness screen

Step 4 — Implement Local Pack Schema

Create local content pack support before AI.

Minimum pack structure:

```
packs/
  water-basics/
    manifest.json
    cards/
      finding-water.md
      purification.md
      storage.md
```

Step 5 — Implement Guided Water Flow

First scenario:

"I'm thirsty / I need water"

Non-AI version should:

1. Show current coordinates.
2. Search downloaded local resource data if available.
3. If no resource pack exists, say it is missing.
4. Show water-finding guide card.
5. Show purification guide card.
6. Show storage guide card.
7. Show source/provenance.

Step 6 — Add Build-in-Public Materials

Create:

- First announcement draft
- Weekly update template
- GitHub issue labels
- Good-first-issue list
- Public roadmap board

21. First Codex Build Prompt

Use this as the first concrete Codex task:

Build an Android-first Kotlin app skeleton for an offline-first open-source survival companion.

Core product concept:

- Four tabs: Map, Tools, Guide, Packs.
- Fully offline-first.
- No account system.
- No cloud dependency.
- No AI in MVP.

MVP requirements:

1. Map tab:

- Request location permission.
- Show current latitude/longitude.
- Show accuracy and altitude if available.
- Allow user to save current location as a waypoint.
- Allow waypoint name, category, and notes.
- Show saved waypoint list.
- Calculate distance and bearing from current location to selected waypoint.
- Add breadcrumb recording start/stop controls.

- Store waypoints and breadcrumbs locally.
2. Tools tab:
 - Show coordinate card.
 - Add timer.
 - Add checklist shortcut.
 - Add emergency mode placeholder.
 - Add flashlight action placeholder.
 3. Guide tab:
 - Load local survival guide cards from bundled sample pack.
 - Support basic local search using SQLite FTS or equivalent.
 - Show card title, category, steps, and provenance/source metadata.
 - Include starter cards for water finding, water purification, water storage, shelter, fire, navigation, signaling, and battery conservation.
 4. Packs tab:
 - Show installed packs.
 - Read pack manifest JSON.
 - Display pack title, version, license, source, category, and installed size if available.
 - Add local import placeholder.
 - Add delete/disable placeholder.
 5. Storage:
 - Use local SQLite for waypoints, notes, breadcrumbs, checklists, installed packs, and guide-card index.
 - Keep schema documented.
 6. Docs:
 - Create README.md.
 - Create ROADMAP.md.
 - Create docs/architecture.md.
 - Create docs/pack-schema.md.
 - Create docs/offline-readiness.md.
 7. Constraints:
 - Do not implement AI yet.
 - Do not implement full offline routing yet.
 - Do not build a full map engine yet.
 - Do not require internet.
 - Make the architecture ready for future RAG and future OpenStreetMap-derived resource packs.

22. Open Questions / Unresolved Decisions

Product/Branding

- Final app name not chosen.
- Final repo name not chosen.
- Whether this sits under Unplugged AI branding or a separate FOSS survival brand is unresolved.

Platform

- Android-first is decided.
- Kotlin/native Android is likely but not formally locked.
- Whether to support iOS later is unresolved.

Map Rendering

- Whether to include an actual map renderer in MVP is unresolved.
- Possible first move is no full map rendering, just coordinates/waypoints/backtrack.
- Later map-rendering library not selected.
- Offline vector map strategy not finalized.

OSM Data Pipeline

Unresolved:

- How to preprocess OSM data.
- Which region format to use.
- Whether to generate packs internally.
- Whether to use MBTiles, PBF, SQLite, or another format.
- How to package POI/resource indexes.
- How often to update map/resource packs.

Spatial Index

Unresolved:

- Exact local spatial index implementation.
- R-tree inside SQLite is likely.
- Need evaluate Android-compatible geospatial libraries.

RAG

Unresolved:

- Whether to start with FTS-only retrieval or vector retrieval.

- Embedding model not chosen.
- Local LLM not chosen.
- Whether model runs on phone, LAN host, or both.
- How to evaluate retrieval quality.
- How to prevent unsupported model output.

Content Licensing

Unresolved:

- Which content sources are safe to include.
- Which licenses are compatible.
- Whether curated packs are official only.
- Whether community content can be accepted.

Open-Source License

Unresolved:

- Apache-2.0
- MIT
- GPL
- AGPL
- Dual license

Earlier strategic thought:

- Permissive license may help adoption.
- Copyleft may better protect openness.
- Final choice not made.

Community Strategy

Unresolved:

- Whether to start with GitHub only or GitHub + Twitter/X immediately.
- Whether to open Reddit announcement early or after first prototype.
- Whether to create Discord later.
- Whether to create a website/docs site early.

23. Things Codex Should Not Assume

Codex should **not** assume:

1. The app is AI-first.

2. The MVP needs a chatbot.
 3. The MVP needs a local LLM.
 4. The MVP needs full offline routing.
 5. The MVP needs full topographic map rendering.
 6. The MVP should include medical advice.
 7. The app should bundle all map data.
 8. The app should share official packs peer-to-peer.
 9. The model should invent map/resource answers.
 10. RAG and map search are the same thing.
 11. FTS alone is enough for nearest-resource search.
 12. GPS requires internet.
 13. OpenStreetMap guarantees accurate survival-critical resources.
 14. The base install should be large.
 15. The app should compete directly with Organic Maps/OsmAnd first.
 16. The project already has implemented files or scripts.
-

24. Key Design Principle

The final product should be built around this principle:

The app should remain useful when the phone has no signal, no account, no cloud, no model loaded, and no internet.

AI is optional.

Maps and survival workflows are the foundation.

25. Exact Plan We Were Converging Toward

The plan is:

1. Launch a public open-source repo.
2. Build in public with weekly updates.
3. Start Android-first.
4. Build a lightweight four-tab app:
 5. Map
 6. Tools
 7. Guide
 8. Packs
 9. Make the MVP non-AI.

10. Implement offline location features:
11. GPS coordinates
12. Saved waypoints
13. Bearing/distance
14. Breadcrumb/backtrack
15. Add pack system:
16. Local JSON/Markdown packs
17. Manifest metadata
18. License/source/provenance
19. Add survival guide cards:
20. Water
21. Fire
22. Shelter
23. Navigation
24. Signaling
25. Battery conservation
26. Add local search over guide cards.
27. Add OpenStreetMap-derived resource packs later.
28. Add geospatial search:
29. nearest water
30. nearest roads/trails/shelters/resources
31. Add guided workflows:
32. "I need water"
33. "I'm lost"
34. "I need shelter"
35. Later add RAG:
36. retrieve guide cards
37. retrieve local resources
38. inject geolocation context
39. cite sources
40. use optional local model

41. Later add field notes:
 42. local notes
 43. location-attached notes
 44. peer-to-peer sharing
 45. signed notes eventually
 46. Later add marketplace-style pack registry.
 47. Avoid medical-heavy content until trust/provenance/review systems are mature.
-

26. Strategic Summary

The best version of this app is not simply:

“Offline survival chatbot.”

It is:

“An offline survival operating layer with maps, location-aware resource search, guided survival workflows, local packs, and provenance-first trust UX.”

The first public build should prove the foundation:

- Small app
- Clean UX
- Offline-ready
- Useful without AI
- Clear pack architecture
- Clear roadmap
- Easy for open-source users to understand and contribute

The breakthrough feature is not one single component.

The breakthrough is the integration of:

- Offline maps/location
- Local resource search
- Survival guide workflows
- Source/provenance UI
- User-controlled packs
- Later RAG/local AI
- Later field-note sharing

That is the wedge.